PENSJONISTENE
digital agency

Setup Guide

Contents

1 Introduction **3**

2 Code **4**
2.1 Setting up flask project . . . 4
2.1.1 Project structure . . . 4
2.1.2 Creating a virtual environment for installation of project dependencies . . . 4
2.1.3 Installing project dependencies . . . 5
2.2 Running app . . . 5
2.2.1 Development mode . . . 5
2.2.2 Testing mode . . . 5
2.2.3 Production mode . . . 5
2.3 Setting up Scrum in GitLab . . . 6
2.3.1 Step 1: Setting iteration cadence . . . 6
2.3.2 Step 2: Preparing labels . . . 9
2.3.3 Step 3: Setting up epic boards . . . 9
2.3.4 Step 4: Creating epics . . . 12
2.3.5 Step 5: Adding issues . . . 13
2.3.6 Step 6: Managing a backlog and sprint . . . 15

3 Commit **19**
3.1 Installing and configuring Git . . . 19
3.2 SSH-keys . . . 19
3.2.1 Generating an SSH key with Git bash . . . 19
3.2.2 Adding key to GitLab . . . 19
3.2.3 Errors with ssh-agent . . . 19
3.3 Cloning project to local machine . . . 19

4 Pre-deploy **21**
4.1 Triggering merge requests . . . 21
4.2 GitLab workflow . . . 21
4.2.1 Stage: test . . . 21
4.2.2 Stage: build . . . 22
4.2.3 Stage: publish . . . 23

5 Deploy **25**
5.1 Kubernetes . . . 25
5.1.1 Kubectl Configuration . . . 26
5.1.2 Talos Configuration . . . 26
5.1.3 Namespace Setup . . . 27
5.1.4 Creating Secrets . . . 27
5.1.5 Initial Database Setup . . . 28
5.1.6 Deploying PostgreSQL with `StatefulSet` . . . 29
5.1.7 PostgreSQL Service . . . 31
5.1.8 Authenticating to the GitLab Container Registry . . . 32
5.1.9 Deploying the Flask Application . . . 34
5.1.10 Exposing the Flask Application . . . 36

5.1.11 Verifying the Flask Deployment 36
5.2 FluxCD 37
5.2.1 Installing Flux 38
5.2.2 Configuring Flux 38
5.2.3 gitlab-auth.yaml 38
5.2.4 gitrepository.yaml 38
5.2.5 kustomization.yaml 39

6 Monitoring **40**
6.1 Installing K9s 40

1 Introduction

This setup guide has been written to aid team members in using the configuration files supplied in the "flask" project solution.

Chapters follow the overall DevOps pipeline as follows:

1. **Code**: the processes developers take to implement changes to the codebase locally
2. **Commit**: pushing changes to the remote repository
3. **Pre-deploy**: the processes that take place when changes have been pushed to the remote repository
4. **Deploy**: the processes that ensure implemented changes are reflect in the live product
5. **Monitoring**: processes that the team use to keep track of the live product

2 Code

2.1 Setting up flask project

2.1.1 Project structure

The structure of the codebase is as follows:

- **Folders**
  - **k8s**: Kubernetes configuration files (for more details see section 5)
  - **project**: Flask app files
    - **database_file**: database files used for sqlite3 database when running application in development mode
    - **image_pool**: location for uploaded images
    - **static**: location of static resources (such as CSS files, images and JavaScript files used in application)
    - **templates**: location for templates used by application functions to render user views
  - **tests**: configuration files for tests and written tests
    - **unit**: where all unit tests are found
    - **integration**: where all integration tests are found
    - **e2e**: where all E2E tests are found
  - **venv**: virtual environment files
- **Files in root directory**
  - **.dockerignore**: file used by Docker to determine which files/folders are included in the Docker build process
  - **.gitlab-ci.yml**: GitLab workflow for CI/CD
  - **app.py**: The entry point for the project
  - **Dockerfile**: The Docker build file used by GitLab workflow
  - **requirements.txt**: Project dependencies
  - **run_test_server.py**: Test entry point used for E2E testing.

2.1.2 Creating a virtual environment for installation of project dependencies

To avoid installing project dependencies globally, one can use a virtual environment. See `https://docs.python.org/3/library/venv.html` for instructions on creating and activating a virtual environment.

2.1.3 Installing project dependencies

Project dependencies are listed in `requirements.txt`. To install project dependencies, run the following command:

```
pip install -r requirements.txt
```

2.2 Running app

2.2.1 Development mode

The application can be ran in development mode using an sqlite3 database. To run the app in development mode, open the terminal, navigate to the root directory and run the following command:

```
python app.py
```

The needed configuration for development mode is given in `./project/config.py` and `./project/__init__.py` which implements the functions to be ran for each URL and solution function.

2.2.2 Testing mode

Testing mode configures the app for testing (see process guide on how to run various automatic tests). A server specific for testing is provided and can be ran with the following command:

```
python run_test_server.py
```

The necessary configuration for testing mode, including a temporary database and upload destination, can be found in `./run_test_server.py`

2.2.3 Production mode

The application is run in production mode on a kubernetes cluster. Production mode is triggered by passing the value "production" to the application through the environment variable "FLASK_ENV". An URL to the PostgresSQL database is also passed to the application.

For more information about running application in production, see section 5.

2.3 Setting up Scrum in GitLab

GitLab includes built-in support for agile project management using the Scrum framework. This allows development teams to plan and execute work using iterations, issue boards, milestones, and epics – all integrated into the same platform.

In this section, it will be described how to set up a practical Scrum workflow in GitLab, including:

- Setting iteration cadence (sprint cycles)
- Adding labels and priorities that support filtering
- Creating epics and issues for tracking user stories
- Configuring issue boards for backlog and sprint planning

This setup provides a complete foundation for managing development in short, focused iterations while maintaining full traceability from planning to deployment.

2.3.1 Step 1: Setting iteration cadence

In order to plan sprints, which is the end goal of this setup, there needs to be an iteration cadence. These are set up by going to the **Group** section in GitLab.

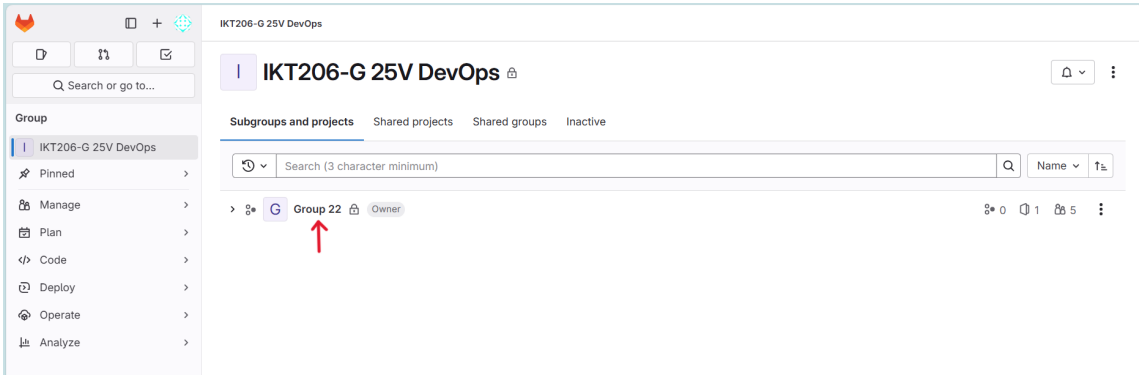

Figure 1: Go to the Group section in GitLab

When there, select **Plan->Iterations**.

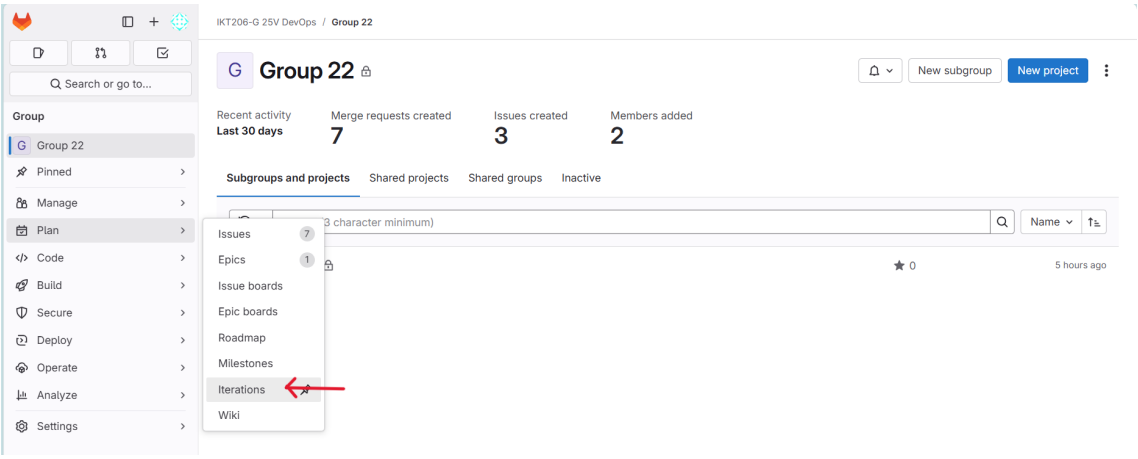

Figure 2: Getting to iterations in the planning section

In iterations, select **New iteration cadence**.

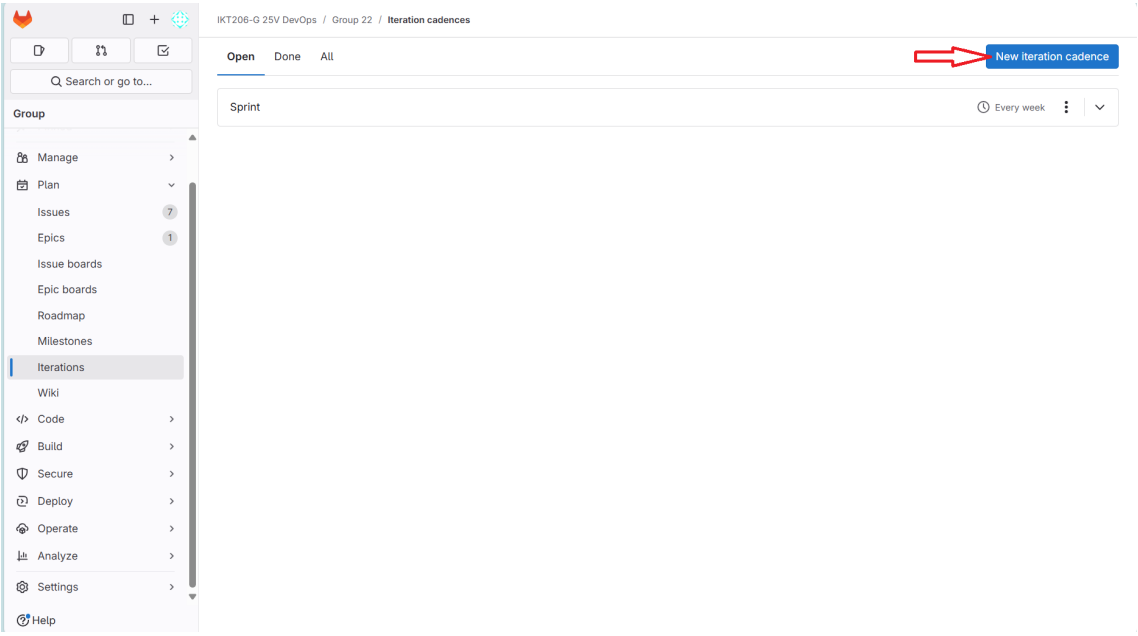

Figure 3: Creating new iteration cadence

When prompted, fill in the fields to describe the new cadence, like title, start date, duration and how many to schedule at once. When finished, press **Create cadence**.

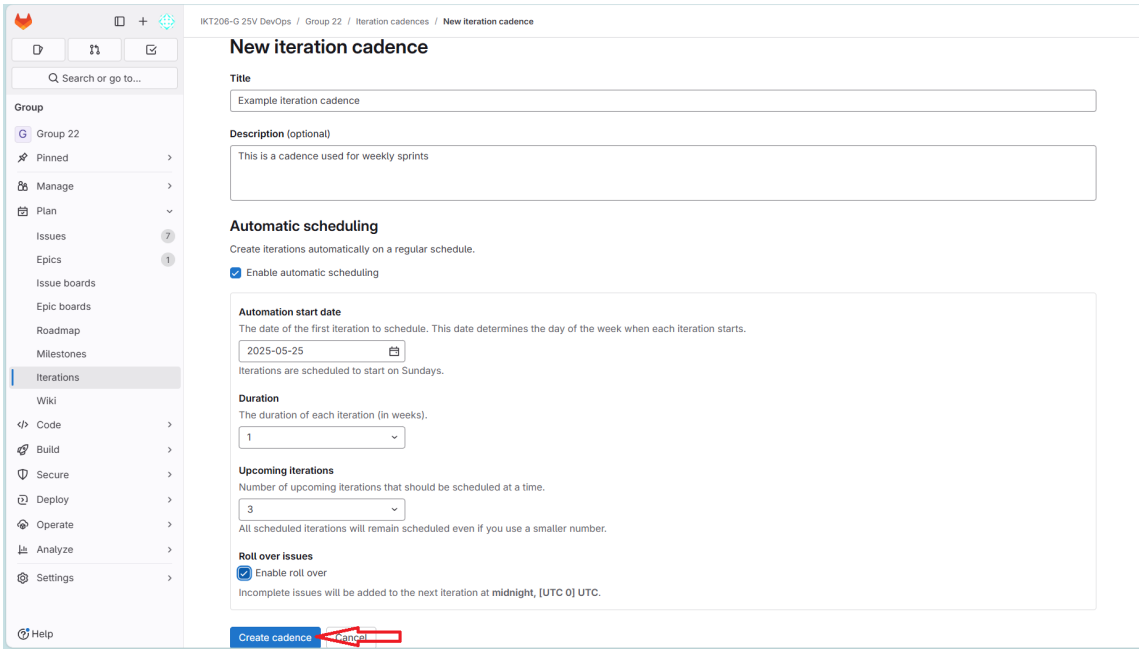

Figure 4: Fill in info and create new cadence

This will later be used to create an **Iteration list** that can be found in the project's issue board.

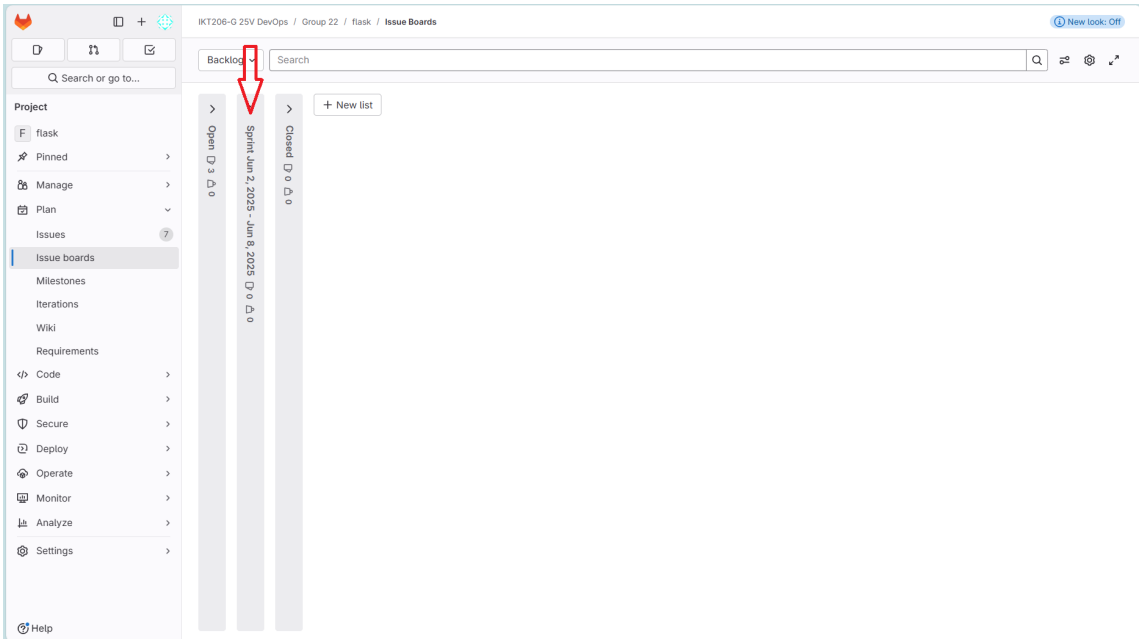

Figure 5: Where the iterations will show up in the issue boards

2.3.2 Step 2: Preparing labels

In order to track work progress, and filter issues and epics, there needs to be labels in place. These can be added by going to the **Group** section in GitLab and selecting **Manage->Labels**.

Select **New label**, which will open a new window where details about the new label can be added.

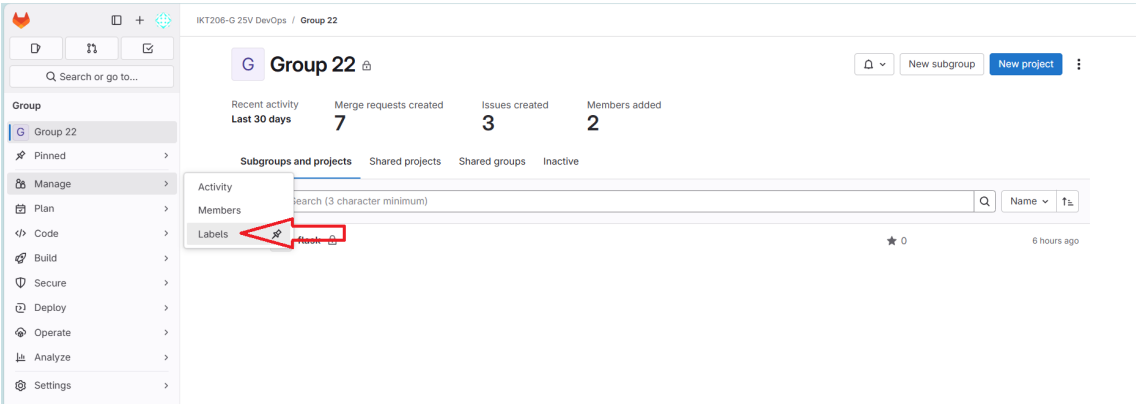

Figure 6: Getting to the labels section

Add the name of the label. Use any labels that will help structure the work being done. Finally, press **Create label**.

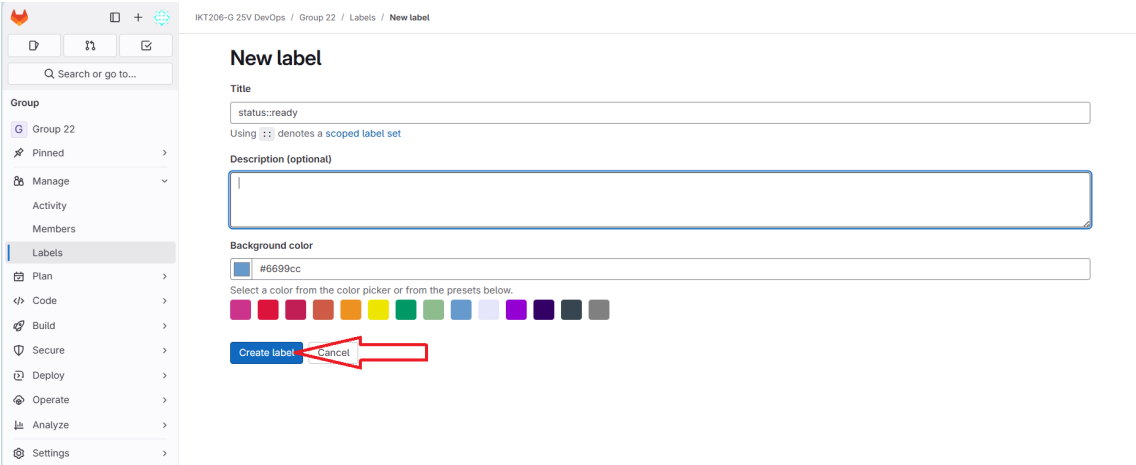

Figure 7: Fill in label details and add it

2.3.3 Step 3: Setting up epic boards

The project's overall process can be developed and maintained in the epic boards.

This is done by going to the **Group** section in GitLab and selecting **Plan->Epic boards**

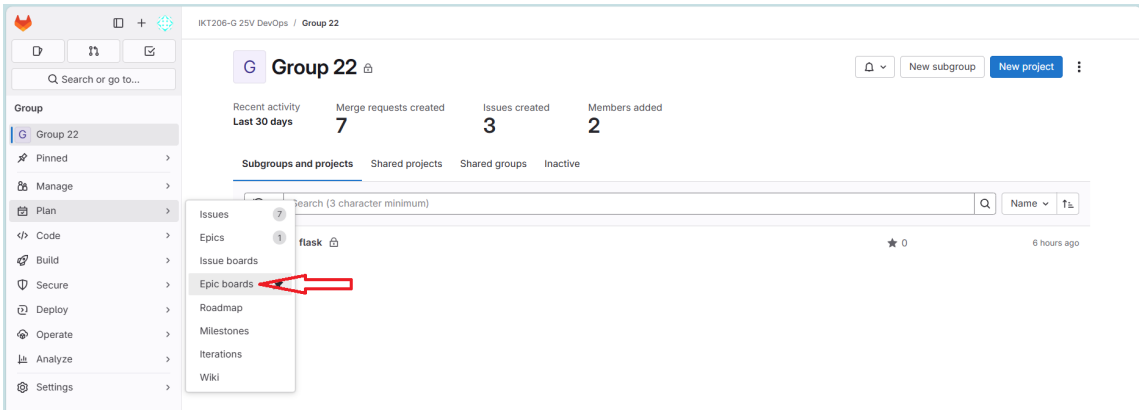

Figure 8: Get to the epic boards

In the top left corner, select the drop down menu and choose **Create new board**.

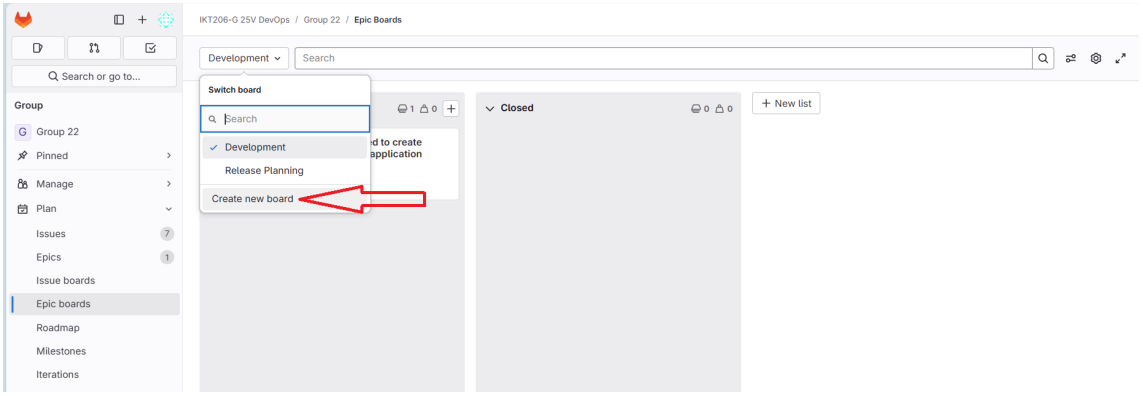

Figure 9: Create new board

Select a suitable name, like *Development*, or *Release planning*. Choose a name that suits the team's needs. Proceed to select **Create board**.

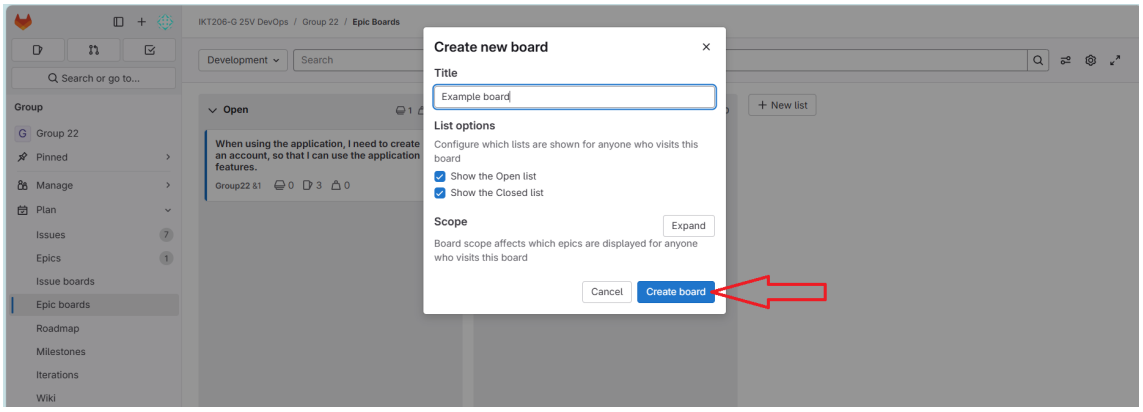

Figure 10: Fill in the name of the new board and create it

The new board will become visible in the drop down menu, and can be selected to view the epics when they have been created.

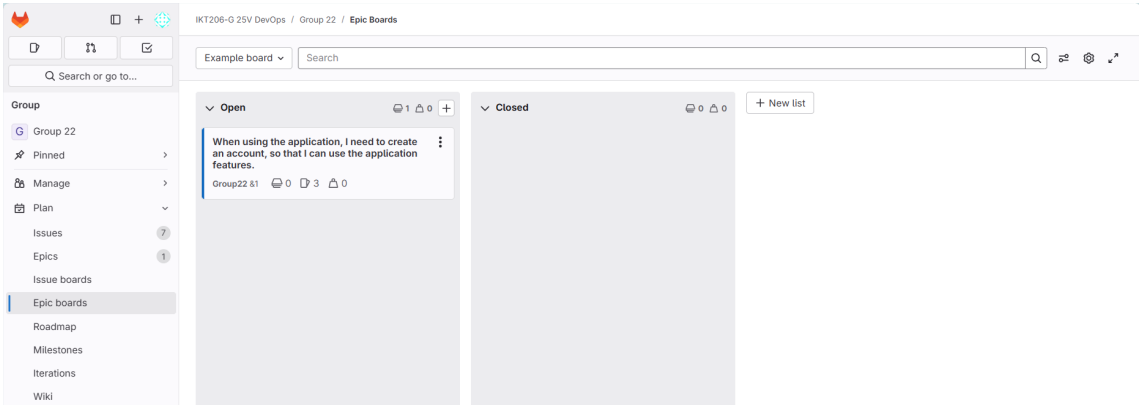

Figure 11: Epic board with an example epic added

In order to track the status of the epic, select the **New list** in the top right corner in the list view. From here, select a fitting label for the new list and press **Add to board**. The labels *priority::now*, *priority::later* and *priority::next* has been added for example purposes. The team should choose suitable labels for their own needs.

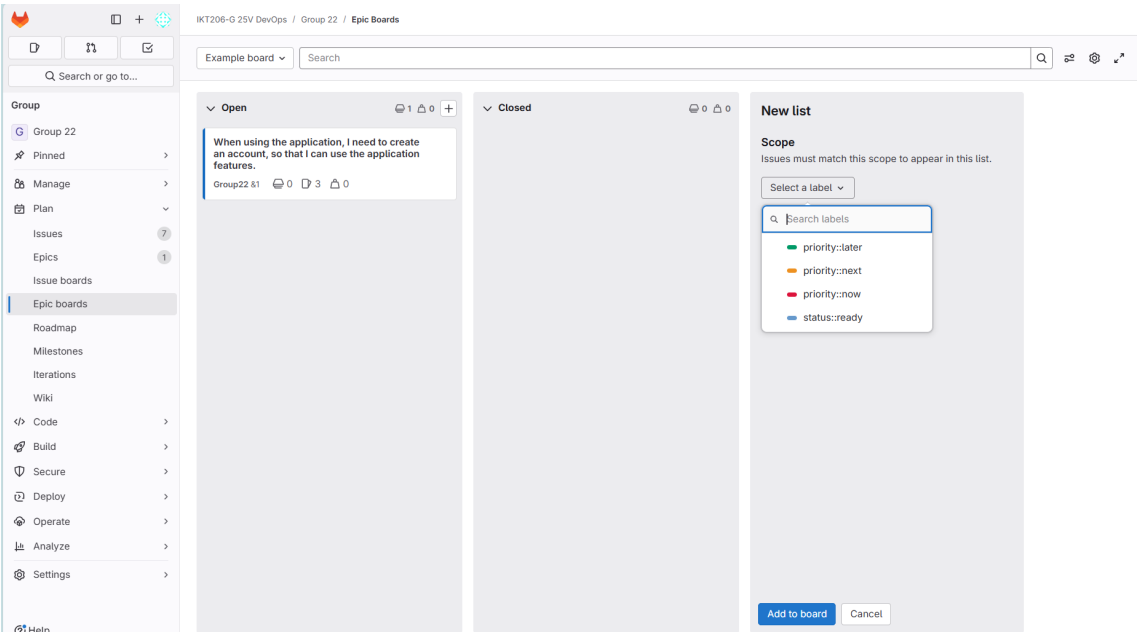

Figure 12: Adding new list based on the previously created labels

After this is done, the new list will appear in the epic board, where the epics can be dragged into to keep track of their status.

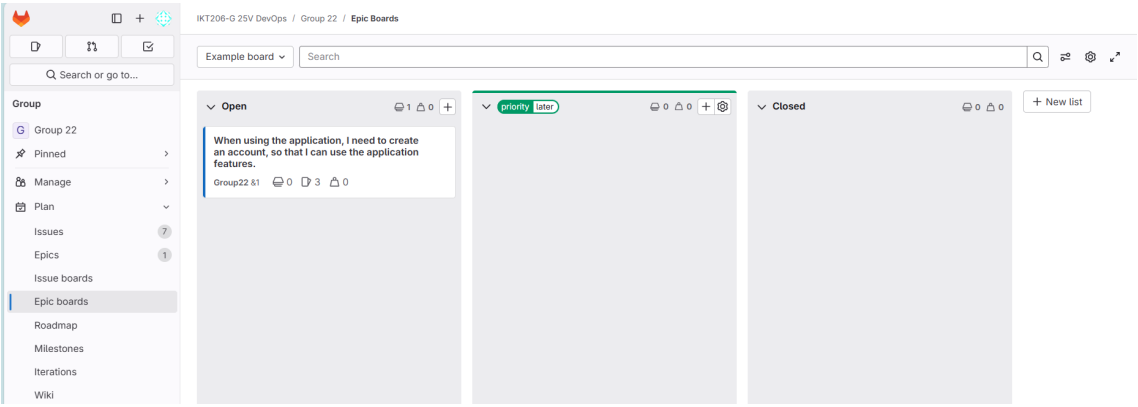

Figure 13: Epic board with status list

2.3.4 Step 4: Creating epics

In order to add issues at a later stage, there needs to be epics to act as parent object. These can be views as *features* as well. In order to add them, go to the **Group** section in GitLab and select **Plan->Epics**. Select **New epic**.

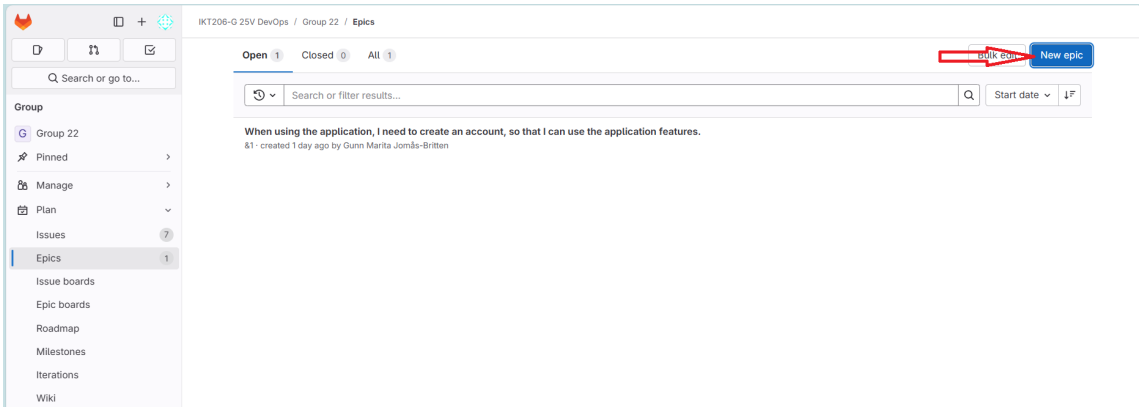

Figure 14: Adding a new epic

This will open a model that needs to be filled out with the relevant information. Fill this out and press **Create epic**.

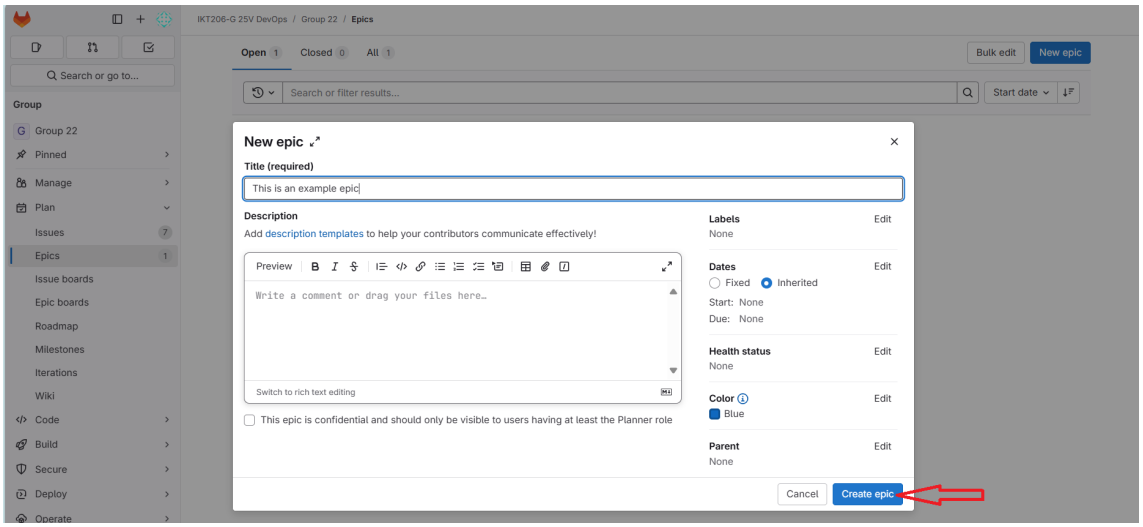

Figure 15: Creating a new epic

The epics can we viewed and organized in the **Epic boards** section under **Plan**.

2.3.5 Step 5: Adding issues

The epic (or feature) can be broken into issues (or user stories). This is done by selecting an epic. This can be done from **Plan->Epics** or **Plan->Epic boards**. This is all done in the **Group** section in GitLab.

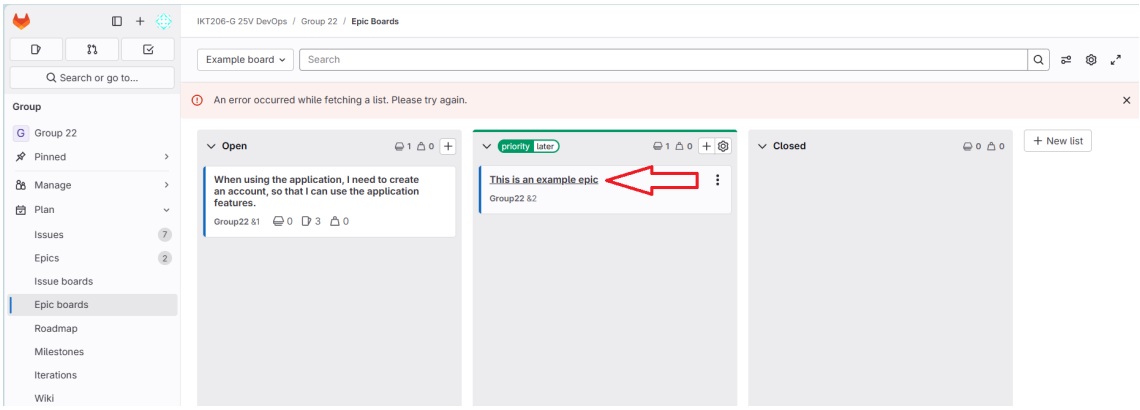

Figure 16: Find the epic/feature that needs issues/user stories

In the epic card, select **Add**, then **New issue**.

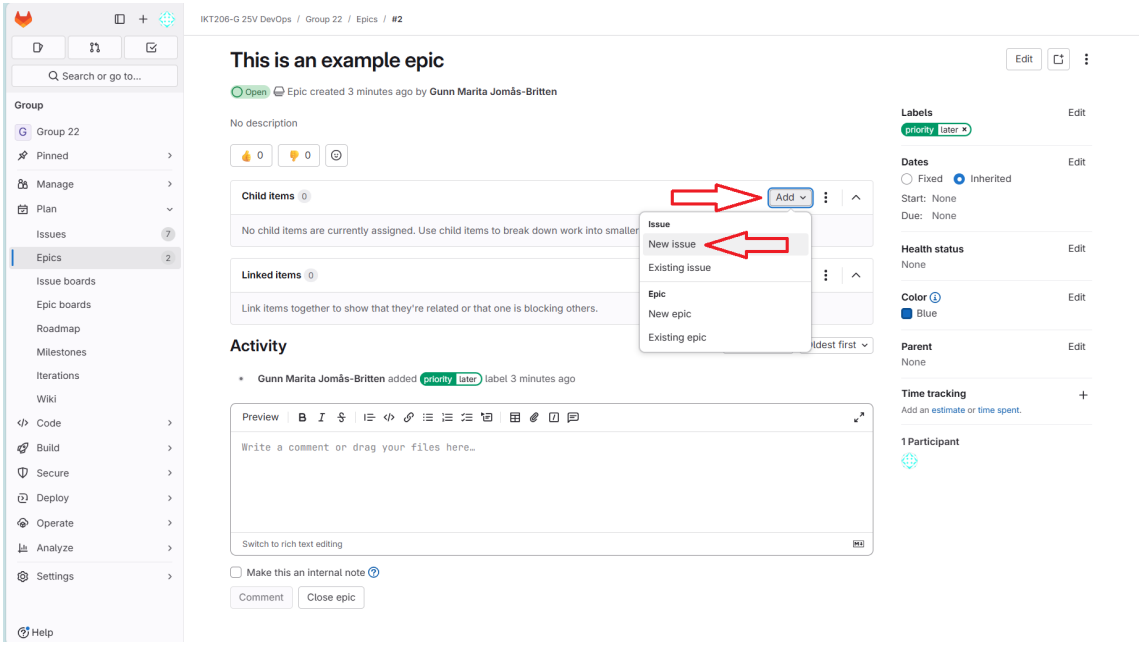

Figure 17: Adding new issue to epic

The previous step will reveal a new *Child item* that needs a title and be connected to a project. Fill out the title and select the project, then **Create issue**.

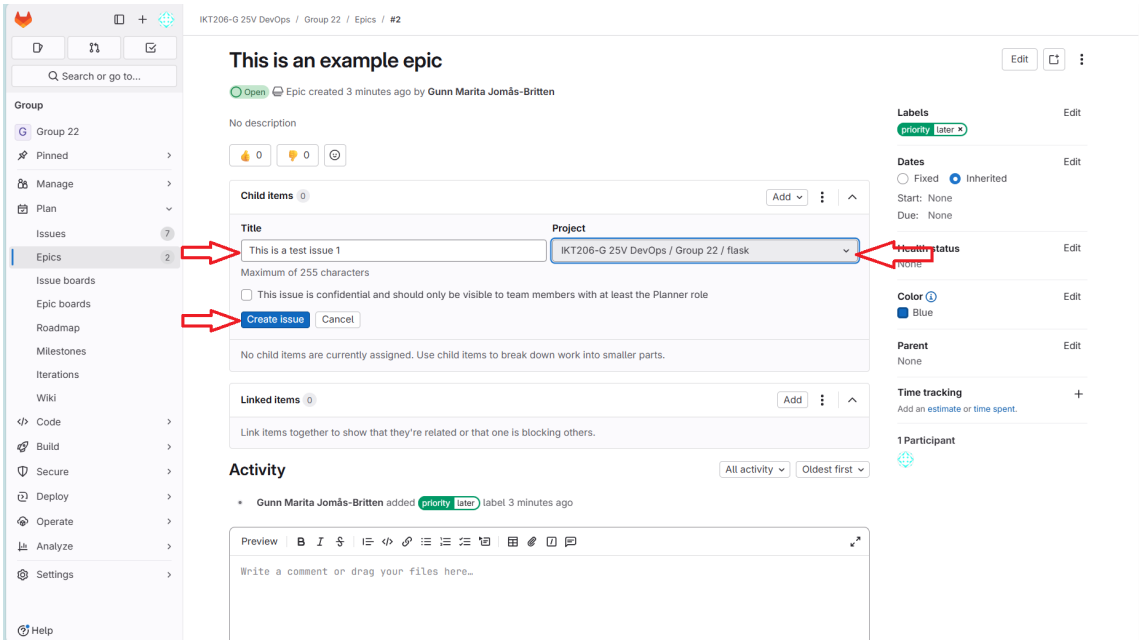

Figure 18: Creating new issue to epic

This can be done multiple times, so an epic can have multiple child issues. This is needed in order to plan a sprint within the project later.

2.3.6 Step 6: Managing a backlog and sprint

When a project is ready to manage a backlog, go to the **Project** section in GitLab. Select **Plan->Issue boards**. This will give an overview of the different boards. If none have been made, select the dropdown menu and **Create new board**. Fill in the fields with a name that suits the needs of the team. In this case, one called *Backlog* has been added. All the issues that has been added under the epics in the previous step, and added under the current project will appear in the *Open* list to the left. The backlog can be managed with lists and filtered by labels.

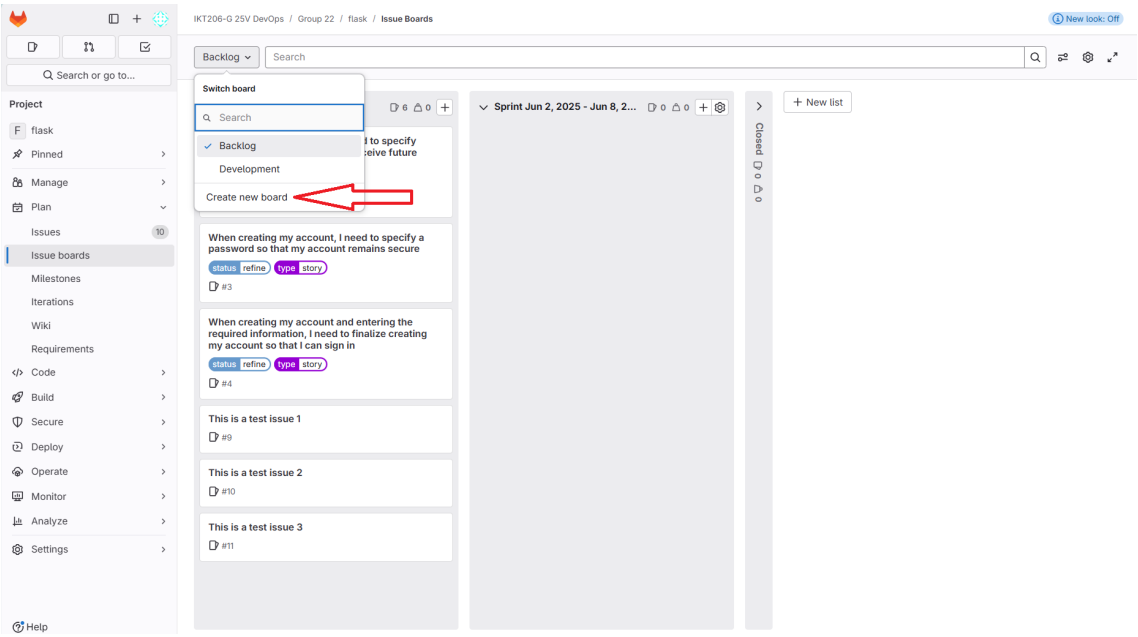

Figure 19: Create a new issue board

Select the appropriate issue board, and then proceed to click **New list** in the top right corner of the list view. Select the **Iteration** option, and select one of the iterations that was created in *Step 1*. This represents a sprint.

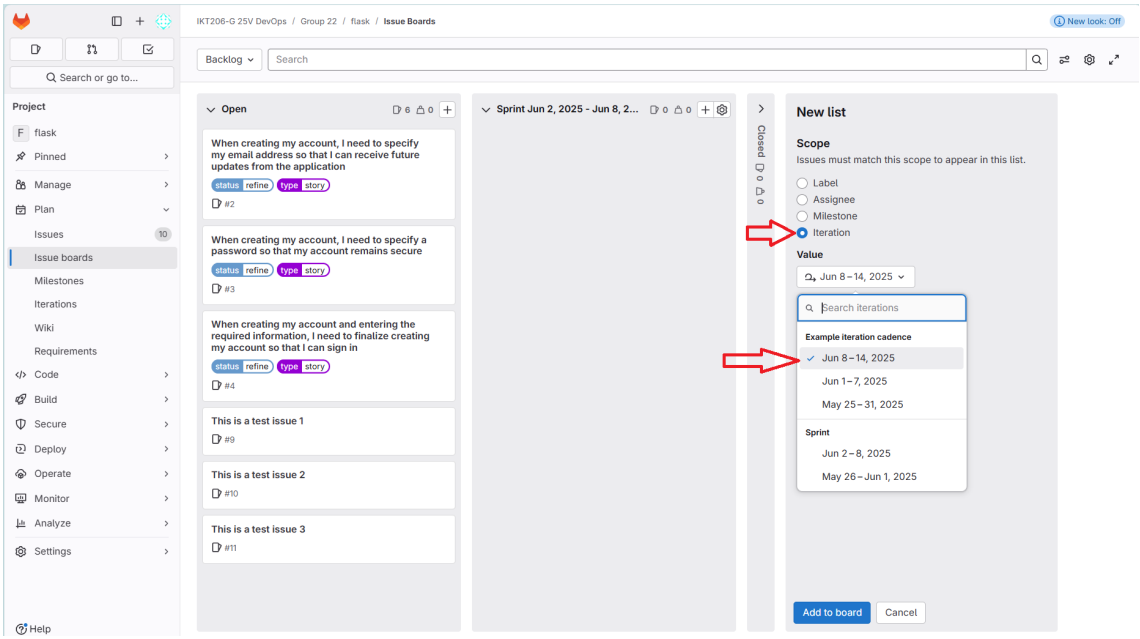

Figure 20: Creating new list based on iteration

When this is done, a new list with the selected iteration will appear. The team can then proceed to drag and drop the issues where they see fit.

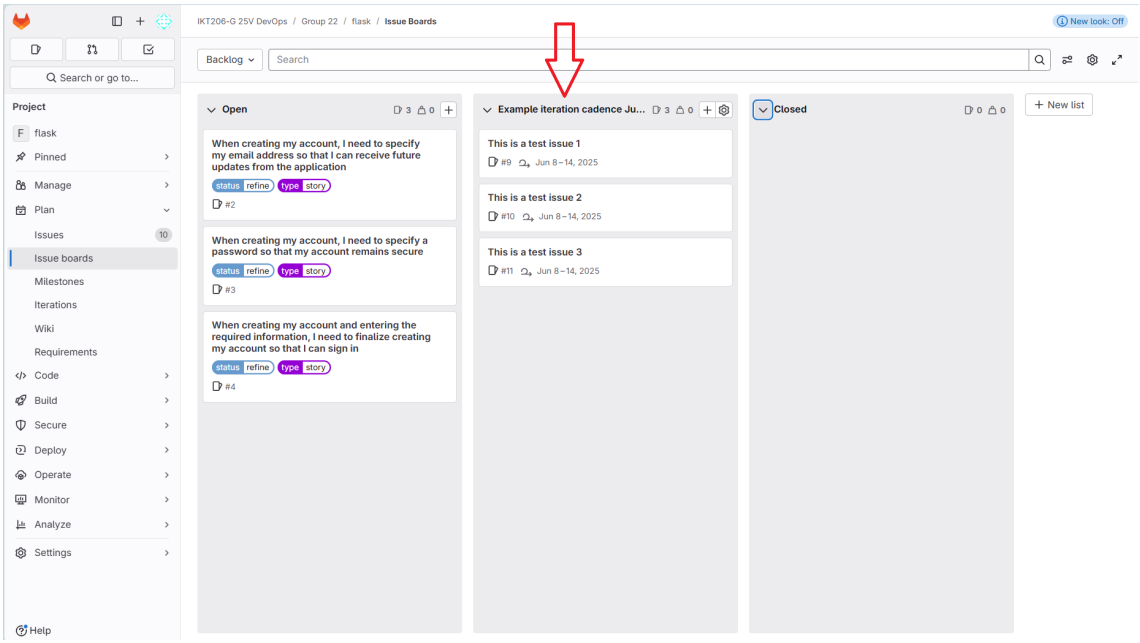

Figure 21: Complete backlog with issues placed in a new sprint

More detailed information can be found in GitLab's documentation:

$https://docs.gitlab.com/tutorials/scrum_events/$

3 Commit

3.1 Installing and configuring Git

Follow the official documentation from Git for installing Git (`https://git-scm.com/book/en/v2/Getting-Started-Installing-Git` [1]) and configuring Git for the first time (`https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup`) [2].

3.2 SSH-keys

3.2.1 Generating an SSH key with Git bash

To access the remote repository, the team member needs to authenticate themselves. This can be done by using a SSH key which can be generated through Git Bash as follows:

1. **Use `ssh-keygen`:**

```
ssh-keygen -t ed25519 -f ~/.ssh/[NAME_OF_KEY] -C "Appropiate comment"
```

2. **Add key to agent**

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/[NAME_OF_KEY]
```

3.2.2 Adding key to GitLab

To add the key to GitLab, follow these steps:

1. **Copy the SSH key**

```
cat ~/.ssh/[NAME_OF_KEY].pub
```

2. **Add key to GitLab**
   (a) Click user icon on GitLab and choose "Preferences"
   (b) Go to "SSH keys" and click "Add new key"
   (c) Paste copied key, assign a title and save the key

3.2.3 Errors with ssh-agent

If experiencing problems with the SSH key, ensure that the SSH-agent is running by using the following command: `eval "ssh-agent -s"`. If an agent pid is returned, the agent is now running.

One may also need to add the key again.

3.3 Cloning project to local machine

To clone a project, run a `git clone` command in Git Bash with the repository URL as given in the example below:

```
git clone git@gitlab.internal.uia.no:ikt206-g-25v-devops/Group22/flask.git
```

The URL for repository can be found on GitLab.

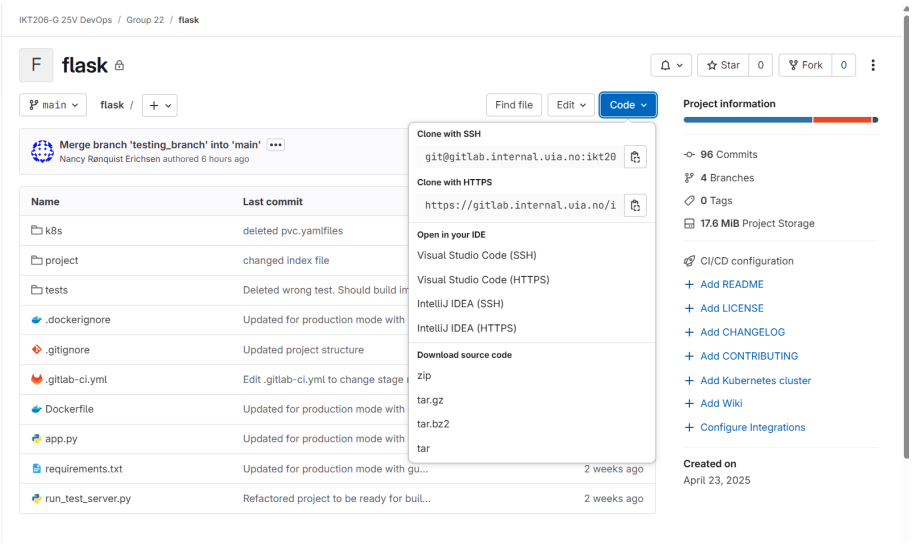

Figure 22: The URL for the repository can be found by clicking "code"

4 Pre-deploy

The GitLab workflow (`./gitlab-ci.yml`) is a key factor for ensuring that pushed changes are applied to the live product. The workflow is automatically triggered when a merge request to the main branch is started. The workflow (pipeline) consists of three stages:

1. test
2. build
3. publish

Each stage depends on the completion of the previous stage.

4.1 Triggering merge requests

The main branch is protected such that only team members with the "Maintainer" role can push directly to it. To push changes to main, a merge request must be used.

Assuming that a team member has created their own local branch, a merge request is triggered by running the following command:

```
    git push \
  -o merge_request.create \
  -o merge_request.target=main \
  -o merge_request.title="[Appropiate title]" \
  -o merge_request.description="[Appropiate description]" \
  -o merge_request.merge_when_pipeline_succeeds \
  -o merge_request.merge_train \
  origin my-feature-branch
```

GitLab is configured to use merge trains [3], placing merge requests in a queue. Each merge request is compared to the earlier merge requests in the queue, thus ensuring they all work together.

On a successful merge, the source branch is automatically deleted.

4.2 GitLab workflow

4.2.1 Stage: test

The test stage runs the three type of tests mentioned in section 2.1.1. Each job in the workflow sets up the needed environment, and runs the tests found in the corresponding directory.

Each test job includes an "artifact" attribute as seen in the code below:

```
artifacts:
when: always
paths:
  - e2e-report.xml
reports:
  junit: e2e-report.xml
```

This allows test results to be saved as an artifact in GitLab. It also provides the functionality to view test results directly in pipeline windows or the merge request as seen in the figures below:

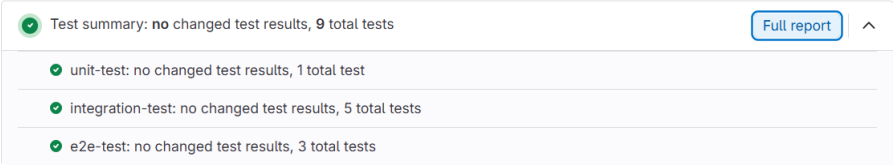

Figure 23: Viewing test results of the tests triggered by a merge request

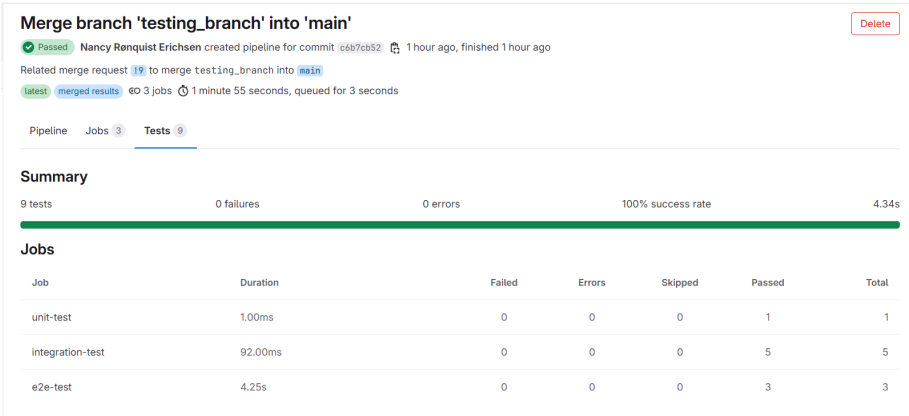

Figure 24: More detailed view of test results. Each record can be clicked on to get more details about individual tests

4.2.2 Stage: build

The build stage is triggered after a merge has taken place. The job uses Kaniko as a tool to build images based on the provided Dockerfile which is shown below.

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# Create working directory
WORKDIR /app

# Install dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

```

```
# Copy requirements file and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code
COPY . .

# Expose the port the app runs on
EXPOSE 5000

# Start the application using Gunicorn
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app:app"]
```

Notice that the port which the flask app communicates through is defined here. Any changes to this must also be reflected in the Kubernetes flask service and deployment files.

Images can be found in the container registry and are labeled with a shortened Git commit hash. The "latest" tag is also updated to point to the latest built image.

4.2.3 Stage: publish

The publish stage of the workflow is used for updating Kubernetes deployments with the newly published image by using the technology FluxCD. See section 5.2 for more information on how FluxCD is used on the Kubernetes cluster.

The following job is for updating the staging namespace with a newly published image:

```
update-staging-manifest:
  stage: publish
  image: alpine:latest
  before_script:
    - apk add --no-cache git sed
    - git config --global user.email "ci2@example.com"
    - git config --global user.name "gitlab-ci"
    - echo ${CI_PUSH_AUTO_KEY}
    - echo ${CI_PUSH_AUTO_USER}
    - git remote set-url origin
    ↪  "https://${CI_PUSH_AUTO_USER}:${CI_PUSH_AUTO_KEY}@gitlab.internal.uia.no/ikt206-g-25v-devops/Group22/

  script:
    - TAG="${CI_COMMIT_SHORT_SHA}"
    - echo $TAG
    - git fetch origin

    # Step 1: Delete remote and local `flux-setup-staging` if they exist
    - git push origin --delete flux-setup-staging || echo "Remote branch did not exist"
    - git branch -D flux-setup-staging || echo "Local branch did not exist"

    # Step 2: Create fresh `flux-setup-staging` from `main`
    - git checkout -b flux-setup-staging origin/main

```

```
    # Step 3: Update image tag
    - echo "Updating image tag to $TAG"
    - 'sed -i "s|image: .*|image: registry.internal.uia.no/ikt206-g-25v-devops/group22/flask:$TAG|" k8s/staging/flask/flask-deployment.yaml'

    # Step 4: Commit and push
    - git add k8s/staging/flask/flask-deployment.yaml
    - 'git commit -m "CI: Oppdaterer image-tag til $TAG for staging" || echo "No changes"'
    - git push origin flux-setup-staging
  rules:
    - if: '$CI_COMMIT_BRANCH == "main" && $CI_PIPELINE_SOURCE == "push"'
      when: always
    - when: never
```

Notice that a dedicated branch called "flux-setup-staging" is used. To avoid merge conflicts, the branch is first deleted and created again based on the current state of the main branch. After this, the string containing the link to the image in the container registry is updated with information about the latest published image, and changes are pushed to the branch.

See section 5.2 for more details regarding how FluxCD updates the deployments after this job is completed.

The same process is completed for the production namespace.

5 Deploy

5.1 Kubernetes

A Kubernetes cluster has been assigned, and access is provided via a distributed `kubeconfig` file. The cluster is already set up and operational.

Unless otherwise noted, all terminal examples in this setup guide assume a Linux-based shell environment. This primarily affects commands related to the file system and shell scripting. Platform-independent tools such as `kubectl`, `talosctl`, and `k9s` work consistently across operating systems, provided they are installed and configured correctly. Equivalent commands can be used on other operating systems (such as macOS or Windows with WSL), but they are not shown in this guide.

To support deployment and configuration, a complete set of Kubernetes manifests is included in the project under the `k8s/` directory. The structure separates environments clearly:

- `staging/` – configuration files for the staging environment
- `production/` – configuration files for the production environment

Each environment folder is organized into subdirectories by component:

- `namespace/` – defines the namespace for the environment
- `configfiles/` – secrets and shared configuration
- `postgres/` – PostgreSQL deployment, service, and initialization script
- `flask/` – Flask application deployment and service

Recommended Apply Order

Apply the following files in the listed order to ensure that all dependencies are correctly resolved. The function and content of each file will be described in the upcoming sections of this guide.

Staging

1. `k8s/staging/namespace/namespace.yaml`
2. `k8s/staging/configfiles/postgres-secret.yaml`
3. `k8s/staging/postgres/postgres-init-configmap.yaml`
4. `k8s/staging/postgres/postgres-deployment.yaml`
5. `k8s/staging/postgres/postgres-service.yaml`
6. `k8s/staging/flask/flask-deployment.yaml`
7. `k8s/staging/flask/flask-service.yaml`

Production

1. `k8s/production/namespace/namespace.yaml`
2. `k8s/production/configfiles/postgres-secret-production.yaml`

3. `k8s/production/postgres/postgres-init-configmap-production.yaml`
4. `k8s/production/postgres/postgres-deployment.yaml`
5. `k8s/production/postgres/postgres-service.yaml`
6. `k8s/production/flask/flask-deployment-production.yaml`
7. `k8s/production/flask/flask-service-production.yaml`

Each of these files can be applied using:

```
kubectl apply -f <path-to-yaml>
```

In addition to the manifest files, a `kubeconfig` file and a Talos `config` file are provided to enable access to the cluster and nodes. It is essential that these configuration files are kept secure and are not committed to version control systems such as Git. Store them locally in a secure location and share them only within the team on a need-to-know basis.

5.1.1 Kubectl Configuration

In order to interact with it, the `kubectl` command-line tool must be installed. Installation instructions can be found on the official Kubernetes website: `https://kubernetes.io/docs/tasks/tools/`.

Once `kubectl` is installed, access to the cluster is configured by storing the provided `kubeconfig` file locally (note: this file should *not* be added to version control), and exporting the path using the following command:

```
export KUBECONFIG=/path/to/kubeconfig
kubectl config get-contexts
```

The first command specifies which configuration file `kubectl` should use. The second command lists all available contexts in that configuration file. This is useful for verifying that you are connected to the correct cluster.

5.1.2 Talos Configuration

A `talosconfig` file has also been provided. This configuration file is used by the `talosctl` tool, which enables secure, API-based interaction with Talos-managed Kubernetes nodes. The setup process is similar to that of the `kubeconfig` file.

`talosctl` can be downloaded from the official Talos website: `https://www.talos.dev/v1.10/talos-guides/install/talosctl/`.

After downloading, make the binary executable and move it to a directory included in your system's `PATH`:

```
chmod +x talosctl
sudo mv talosctl /usr/local/bin/
```

Once installed, configure the environment variable so that `talosctl` can locate the configuration file:

```
export TALOSCONFIG=/path/to/talosconfig
```

This environment variable functions similarly to `KUBECONFIG` for `kubectl`, and should point to the local path of the `talosconfig` file.

By configuring this system, the team gain access to the `talosctl` command suite, which allows for secure, low-level management of Kubernetes nodes. This enables tasks like viewing logs, inspecting services, rebooting nodes, and applying OS updates, all without SSH access.

5.1.3 Namespace Setup

To keep the staging and production environments separated in Kubernetes, two dedicated namespaces should be created: `staging` and `production`. Namespaces help logically separate applications, databases, and other resources, so that testing and changes in staging won't interfere with production[4].

The easiest and most consistent way to do this is by using the YAML files provided with the project. These are located in the `k8s/` directory, under the `staging/` and `production/` subfolders.

To create the necessary namespaces, run the following commands from each of these directories:

```
# From k8s/staging/namespace
kubectl apply -f namespace.yaml

# From k8s/production/namespace
kubectl apply -f namespace.yaml
```

Alternatively, the namespaces can be created manually using:

```
kubectl create namespace staging
kubectl create namespace production
```

Using namespaces provides a clear separation between environments, which is a key principle of DevOps.All resources (applications, databases, services) must specify the appropriate `namespace` in their corresponding YAML definitions to ensure that they are deployed to the correct environment.

```
kubectl get namespaces
```

This will display a list of all active namespaces in the cluster, including `staging` and `production` if everything is set up correctly.

5.1.4 Creating Secrets

Before deploying the application and database, environment-specific credentials need to be stored securely in Kubernetes. This is done using `Secret` resources. Each environment—staging and production—should have its own secret file[5].

Each environment folder includes an example file called `postgres-secret.yaml`. This file contains base64-encoded values for things like the database name, user, password, and connection URL:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
```

```yaml
  namespace: staging
type: Opaque
data:
  POSTGRES_DB: <base64-encoded-dbname>
  POSTGRES_USER: <base64-encoded-username>
  POSTGRES_PASSWORD: <base64-encoded-password>
  DATABASE_URL: <base64-encoded-url>
```

Customizing the Secrets

The team must generate its own credentials and encode them using base64. This can be done with the following command in a terminal:

```bash
echo -n "your-value-here" | base64
```

Replace `your-value-here` with the actual database name, username, password, or URL. The resulting base64-encoded value should then be copied into the corresponding field in the appropriate secret YAML file.

Formatting the `DATABASE_URL`

The `DATABASE_URL` must follow the standard PostgreSQL URI format:

```
postgresql://<username>:<password>@<hostname>:<port>/<database>
```

In this setup, the hostname should point to the PostgreSQL pod via the headless service. If the PostgreSQL service is named `postgres` in the `staging` namespace, this hostname will direct traffic to the correct database instance.

A complete example of the `DATABASE_URL` might look like:

```
postgresql://flaskuser:flaskpassword@<hostname>:5432/flaskdb
```

The placeholder `<hostname>` should be replaced with the appropriate service address after the database service has been created. Once the URL is constructed, it should be encoded using:

```bash
echo -n "your-database-url" | base64
```

The resulting encoded value should then be added to the `DATABASE_URL` field in the secret file.

Applying the Secret

Once the secret values have been added, the secret can be applied to the appropriate namespace with the following command:

```bash
kubectl apply -f postgres-secret.yaml
```

It is important to ensure that the `namespace` field in the secret file matches the intended environment. Without these secrets in place, neither the PostgreSQL pod nor the Flask application will be able to start correctly, as they rely on these environment variables for configuration.

5.1.5 Initial Database Setup

To ensure the PostgreSQL database includes the necessary tables and test data on first launch, an initialization script is included in the project.

The configuration file, named `postgres-init-script.yaml`,
is located in the `k8s/<namespace>/postgres` folders. It defines a simple SQL script that sets up three tables and inserts a few predefined users for testing. The full SQL content is embedded in the YAML file. The excerpt below shows the format, including one table definition and a sample insert statement. Additional tables and user entries have been omitted here for brevity.

```yaml
kind: ConfigMap
metadata:
  name: postgres-init-script
  namespace: <namespace>
data:
  init.sql: |
    -- Users table
    CREATE TABLE IF NOT EXISTS users (
        id TEXT PRIMARY KEY,
        pw TEXT NOT NULL
    );

    -- Notes and images tables are also defined here ...

    INSERT INTO users (id, pw) VALUES
      ('ADMIN', '8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918'),
      (... additional users ...);
```

To apply the initialization script to the cluster, the following command should be run from within the appropriate directory:

```
kubectl apply -f postgres-init-script.yaml
```

Ensure that the `namespace` field in the YAML file is correctly set to either `staging` or `production`, depending on the target environment.

This initialization script is executed by PostgreSQL the first time the pod starts, provided the persistent volume is empty. It is essential that the database deployment references this ConfigMap, so that the script is mounted in the correct directory. Details on how to configure this are provided in section 5.1.6.

5.1.6 Deploying PostgreSQL with `StatefulSet`

PostgreSQL is deployed using a Kubernetes `StatefulSet`, which is well-suited for stateful applications such as databases. The deployment configuration is defined in the
`postgres-deployment.yaml` file, with separate versions provided for the `staging` and `production` environments. This setup ensures that:

- PostgreSQL runs as a non-root user, following strict security best practices
- Secrets are used to inject credentials securely as environment variables
- Persistent storage is provisioned using a PersistentVolumeClaim (PVC)
- The initialization script is executed automatically on first startup

The minimal example below highlights the essential components:

```yaml
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  template:
    spec:
      containers:
        - name: postgres
          image: postgres:13
          env: [...] # From Secret
          volumeMounts:
            - name: data
            - name: init-db
      volumes:
        - name: init-db
          configMap:
            name: postgres-init-script
  volumeClaimTemplates:
    - name: data
```

A container that uses environment variables from a Secret, mounts an initialization script from a ConfigMap, and attaches a persistent volume. The complete configuration is available in the provided deployment files.

Key components in the deployment

- `StatefulSet` – ensures consistent pod identity (`postgres-0`) and stable storage
- `volumeMounts` – mounts a persistent volume for data, and mounts the SQL init script from a `ConfigMap`
- `env` – environment variables are populated from a `Secret`
- `volumeClaimTemplates` – automatically requests a PersistentVolume for the pod's data
- `PGDATA` – set to a subdirectory inside the mounted volume, as required by PostgreSQL

Applying the deployment

To create the PostgreSQL deployment, navigate to the appropriate directory and run:

```
kubectl apply -f postgres-deployment.yaml
```

This will start the PostgreSQL database inside the specified namespace. On first startup, if the data volume is empty, the SQL initialization script will be executed automatically. This will create the required tables and insert initial users.

Worth noting

- The `ConfigMap` with `init.sql` must exist before this deployment is applied
- The `Secret` containing database credentials must also be in place

- If you delete the pod but not the PVC, the init script will **not** run again — because PostgreSQL detects existing data
- The `volumeClaimTemplates` section ensures each instance of the database pod has its own persistent volume

Replica Count and Future Improvements

In the current configuration, the PostgreSQL database runs as a single replica. This choice is intentional and aims to keep the setup simple and easy to manage during development and testing[6].

While this setup is sufficient for non-critical environments, it does not offer high availability or fault tolerance. In a production context, relying on a single database instance creates a single point of failure: if the pod crashes or the node becomes unavailable, the database service will be temporarily disrupted[6].

For future iterations, it may be relevant to explore solutions that support database replication, automated failover, or clustering. These approaches involve more complex configuration but are essential in production-grade environments where uptime and data resilience are key concerns.

Although such features may not be necessary at this stage, it is important for the team to understand the limitations of the current setup and consider potential scalability and reliability needs as the application matures.

5.1.7 PostgreSQL Service

To allow other applications (such as Flask) to access the PostgreSQL database, a Kubernetes Service is created. This service acts as a stable network endpoint in front of the PostgreSQL pod.

```yaml
kind: Service
metadata:
  name: postgres
  namespace: staging
  labels:
    app: postgres
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

A headless service is configured by setting `clusterIP: None`. This enables DNS-based access directly to the specific pod within the StatefulSet. For example, the following hostname:

```
postgres-0.postgres.staging.svc.cluster.local
```

resolves to the IP address of the `postgres-0` pod, via the service named `postgres` in the `staging` namespace. This DNS name is used in the `DATABASE_URL` environment variable to allow the Flask application to connect directly to the corresponding PostgreSQL instance.

To create the service that exposes the PostgreSQL pod within the cluster, apply the provided service definition using:

```
kubectl apply -f postgres-service.yaml
```

Ensure that the service file is placed in the appropriate directory and that the `namespace` field is correctly set to match the target environment.

5.1.8 Authenticating to the GitLab Container Registry

Before deploying the Flask application, Kubernetes needs permission to pull the Docker image from the GitLab Container Registry. This is accomplished by creating a Kubernetes `docker-registry` secret that stores access credentials.

First step: Create a Project Access Token in GitLab

Navigate to the teams project in GitLab and go to:

- `Settings` → `Access Tokens`

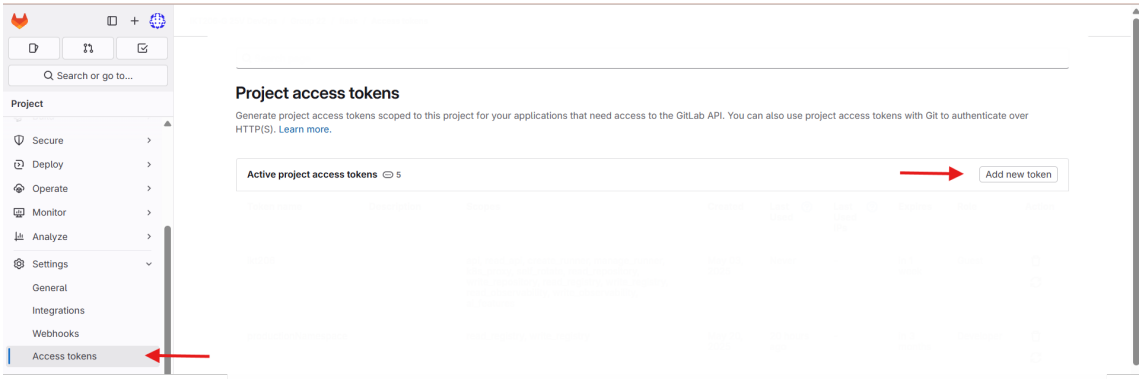

Figure 25: How to add a token

Create a new token with:

- **Role:** Developer
- **Scopes:** `read_registry`
- **Expiration date:** set appropriately

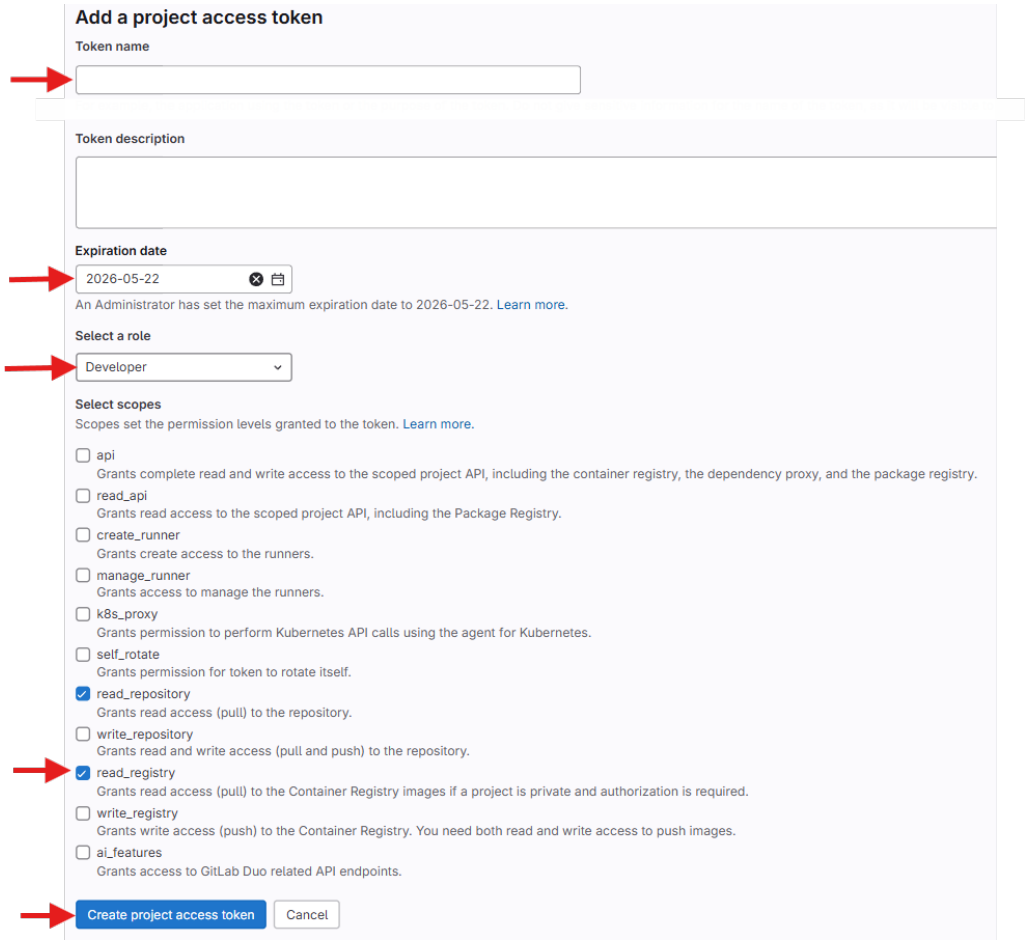

Figure 26: How to make a token

After selecting `Create project access token`, the generated token should be copied immediately, as it will not be displayed again. This token must be stored securely and must never be committed to version control.

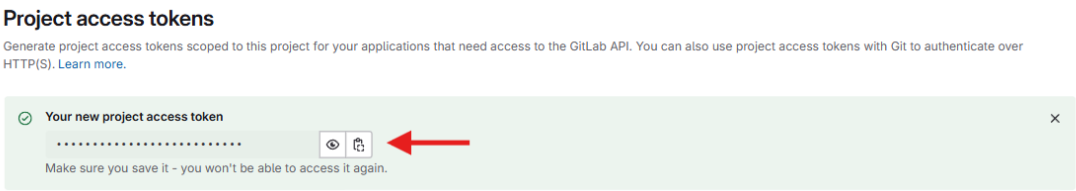

Figure 27: How to save/copy a token

Second step: Create a Kubernetes image pull secret

The token can then be used to create a secret in the cluster by running the following command:

```
kubectl create secret docker-registry gitlab-registry-secret \
  --docker-server=registry.internal.uia.no \
  --docker-username=<gitlab-username> \
  --docker-password=<access-token> \
  --docker-email=<your@email.com> \
  --namespace=<namespace>
```

Replace the following placeholders with the appropriate values:

- `<gitlab-username>` – the name of the relevant GitLab group or user
- `<access-token>` – the access token generated in step 1
- `your@email.com` – an arbitrary email address used for identification purposes
- `<namespace>` – either `staging` or `production`, depending on the target environment

The `-docker-server` flag specifies the address of the container registry where the image is hosted. This is the server Kubernetes will connect to when pulling container images for the Flask application.

Third step: Referencing the secret

Make sure the Flask `Deployment` references the secret under `imagePullSecrets`, like so:

```
spec:
  imagePullSecrets:
    - name: gitlab-registry-secret
```

This ensures that Kubernetes can authenticate and pull the image successfully when deploying the application. How to set up the flask deployment is shown in section 5.1.9

5.1.9 Deploying the Flask Application

The Flask is deployed using a Kubernetes `Deployment` resource. Separate deployment files are provided for each environment (`staging` and `production`) and can be found under

`k8s/<namespace>/flask/`.

The deployment runs three replicas of the Flask API, enabling high availability and load balancing across cluster nodes. The container image is pulled from GitLab's internal registry and requires a Docker registry secret to authenticate.

Key deployment features

- Pulls the container image from the GitLab Container Registry
- Uses a Docker registry secret (`gitlab-registry-secret`)
- Injects the `DATABASE_URL` via a Kubernetes `Secret`
- Exposes port `5000`, the default port used by Flask

- Sets `FLASK_ENV=production` for runtime configuration

Example snippet

The following excerpt illustrates the core structure of the Flask deployment file. Only the most relevant parts are shown. The full file is available in the project repository under `k8s/<namespace>/flask/flask-deploym`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-api
  namespace: staging
spec:
  replicas: 3
  template:
    spec:
      imagePullSecrets:
        - name: gitlab-registry-secret
      containers:
        - name: flask-api
          image: registry.internal.uia.no/.../flask:<tag>
          env:
            - name: FLASK_ENV
              value: "production"
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: DATABASE_URL
```

Replace `<tag>` with the image version tag from your GitLab container registry.

Connecting to PostgreSQL

The Flask application reads the database connection string from the `DATABASE_URL` environment variable, which is injected from a Kubernetes `Secret` applied in the same namespace. The URL should reference the PostgreSQL pod through the headless service.

Example for the `staging` environment:

```
postgresql://flaskuser:flaskpassword@postgres-0.postgres.staging.svc.cluster.local:5432/flaskdb
```

Applying the deployment Ensure that the Docker registry secret and the database `Secret` have been created in the target namespace. Then apply the deployment using:

```
kubectl apply -f flask-deployment.yaml
```

Once deployed, the Flask pods will start and attempt to connect to PostgreSQL using the injected credentials.

5.1.10 Exposing the Flask Application

To make the Flask API accessible from outside the cluster, a Kubernetes `Service` of type `NodePort` is used. This binds a port on each node to the internal Flask service, allowing access from a web browser or HTTP client. Separate service files are provided for `staging` and `production`, located under `k8s/<namespace>/flask/`.

Example snippet

```yaml
apiVersion: v1
kind: Service
metadata:
  name: flask-api-service
  namespace: staging
spec:
  selector:
    app: flask-api
  type: NodePort
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

This maps port `5000` inside the Flask container to port `80` on the service, which is then exposed as `NodePort 30080` on each Kubernetes node. You can reach the Flask API at:

```
http://<node-ip>:30080
```

Replace `<node-ip>` with the address of any cluster node. To list available node IPs, use:

```
kubectl get nodes -o wide
```

Applying the service

Ensure the namespace is correctly set in the file, and that the Flask deployment is already running. Then apply the service using:

```
kubectl apply -f flask-service.yaml
```

5.1.11 Verifying the Flask Deployment

After deployment, verify that the service is operational and accessible.

Check Pod Status

```
kubectl get pods -n <namespace>
```

If any pods fail to start, describe them to inspect potential issues:

```
kubectl describe pod <pod-name> -n <namespace>
```

View Logs from Flask

Check the application logs to confirm proper startup and environment variable loading:

```
kubectl logs deployment/flask-api -n <namespace>
```

Common Issues to Check

- `DATABASE_URL` must be correctly base64-encoded and accessible via a `Secret`
- PostgreSQL must be running and reachable at

  `postgres-0.postgres.<namespace>.svc.cluster.local`
- The `gitlab-registry-secret` must exist in the same namespace
- The host machine or browser must be able to access `<node-ip>:30080`

Accessing and Testing the Flask Interface

The Flask application serves both API and frontend components. Once deployed, open the following in your browser:

```
http://<node-ip>:30080
```

You should see a welcome page with navigation options such as `Home`, `Public`, and `Private`.

Note: the service is exposed on port `30080` in the `staging` namespace. To access the production environment, use the corresponding port defined in the `flask-service` — in this case, `30081`.

Verify Database Connection

To confirm Flask is communicating with PostgreSQL:

1. Perform an action via the Flask UI (e.g., create a user)
2. Access the PostgreSQL pod:

   ```
   kubectl exec -it postgres-0 -n staging -- psql -U <db-username> -d <db-name>
   ```
3. Query the users table:

   ```
   SELECT * FROM users;
   ```

If the new data appears, the connection is functioning correctly.

5.2 FluxCD

In order for Kubernetes to automatically deploy the updated container images built by GitLab CI/CD, a bridge is required between GitLab and the Kubernetes cluster. Since Kubernetes cannot directly monitor GitLab repositories, a GitOps tool is needed to pull changes and apply them to the cluster. For this purpose, **FluxCD** has been selected.

Flux continuously watches a specified Git repository and applies changes to the Kubernetes cluster when new commits are pushed. This enables automated and version-controlled deployments for both staging and production environments.

5.2.1 Installing Flux

FluxCD can be installed in the Kubernetes cluster using Flux CLI. An official installation guide is provided by Flux through their website and can be found here: *https://fluxcd.io/flux/installation/*

To make it easier to use Flux commands in the terminal, add the CLI to the system `PATH`, allowing use of forthright commands like `flux install`. This installs the core Flux controllers in the `flux-system` namespace of the Kubernetes cluster.

5.2.2 Configuring Flux

To configure FluxCD, a set of manifest files must be applied to the cluster:

- `gitlab-auth.yaml` – authentication secret for accessing the GitLab repository
- `gitrepository.yaml` – defines the Git repository and branch Flux should watch
- `kustomization.yaml` – defines which files to apply from the repository, and to which namespace

In this project, we chose to split both `gitrepository.yaml` and `kustomization.yaml` into separate files for **staging** and **production**, to reflect the separation of environments. This results in five files that must be applied to the Kubernetes cluster:

- `gitlab-auth.yaml`
- `gitrepository-staging.yaml`
- `gitrepository-production.yaml`
- `kustomization-staging.yaml`
- `kustomization-production.yaml`

The authentication file remains the same for both environments, while the other files are customized per environment.

5.2.3 gitlab-auth.yaml

The `gitlab-auth.yaml` file defines a Kubernetes secret that allows Flux to access a private GitLab repository using a personal access token. It uses `type: kubernetes.io/basic-auth`, which is supported by FluxCD for basic HTTP authentication.

The username is typically a service account or bot user, and the password is a personal access token (PAT) with `read_repository` scope.

To update the credentials, simply modify the values in the YAML file and re-apply it with:

```
kubectl apply -f gitlab-auth.yaml
```

5.2.4 gitrepository.yaml

This file tells Flux which Git repository and branch to track. For example, in `gitrepository-staging.yaml`, the branch `flux-setup-staging` is used to track deployments to the staging environment.

To change the branch Flux watches, update the `ref.branch` field:

```
spec:
  ref:
    branch: flux-setup-staging
```

Make sure the specified branch exists and contains the expected Kubernetes manifests.

5.2.5 kustomization.yaml

This file defines what resources Flux should apply to the cluster, from which path in the Git repository, and to which namespace.

Each environment (staging and production) has its own `kustomization.yaml` file, pointing to separate folders like:

```
spec:
  path: ./k8s/staging/flask
  targetNamespace: staging
```

The team can update the path or namespace if the folder structure or deployment targets change.

6 Monitoring

To monitor and interact with the Kubernetes cluster, the following tools are required:

- `kubectl` – for interacting with Kubernetes resources
- `talosctl` – for accessing and debugging Talos-managed nodes
- `k9s` – for live monitoring via a terminal-based UI

Installation and configuration steps for `kubectl` and `talosctl` are described in Section 5.1.1 and 5.1.2. Once configured, these tools can be used as described in the process guide.

6.1 Installing K9s

K9s is a terminal-based user interface for interacting with Kubernetes clusters. It provides an efficient way to navigate, monitor, and manage deployed resources. K9s continuously observes the state of the cluster and presents relevant information and commands to interact with various Kubernetes objects, helping streamline operational workflows [7]. K9s can be installed by downloading the appropriate version for the target operating system:

- Follow the installation guide at their homepage: https://k9scli.io/topics/install/
- Extract the binary and move it to a directory in the system's `PATH`, such as `/usr/local/bin`
- Confirm the installation by running:

  ```
  k9s version
  ```

Once installed, K9s will use the currently active Kubernetes context (as defined by `kubectl`). No further configuration is required. For usage instructions, see the process guide under the monitoring section.

References

[1] *Git - Installing Git*, [Online; accessed 18. May 2025], May 2025. [Online]. Available: https://git-scm.com/book/en/v2/Getting-Started-Installing-Git.

[2] *Git - First-Time Git Setup*, [Online; accessed 18. May 2025], May 2025. [Online]. Available: https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup.

[3] *Merge trains | GitLab Docs*, [Online; accessed 23. May 2025], May 2025. [Online]. Available: https://docs.gitlab.com/ci/pipelines/merge_trains.

[4] Y. Örcün, *The Importance of Kubernetes Namespace Separation*, [Online; accessed 21. May 2025], May 2025. [Online]. Available: https://kubeops.net/blog/the-importance-of-kubernetes-namespace-separation.

[5] *Secrets*, [Online; accessed 21. May 2025], Nov. 2024. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/secret.

[6] *Run a Single-Instance Stateful Application*, [Online; accessed 22. May 2025], Aug. 2023. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/run-single-instance-stateful-application/?utm_source=chatgpt.com.

[7] *K9s - Manage Your Kubernetes Clusters In Style*, [Online; accessed 23. May 2025], Apr. 2025. [Online]. Available: https://k9scli.io.